Mock Paper 2 — Algorithms & Programming (H446/02) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

> Marking guidance: award marks for valid alternative wording that conveys the same point. Code answers may be in OCR Exam Reference Language or any high-level language; award marks for correct logic regardless of syntax dialect. Trace marks require the working to be shown, not just a final answer.

Question 1 — Computational thinking [12]

(a) [4] — 1 mark each, max 4: - Representational abstraction = removing/hiding unnecessary detail to form a model that keeps only the details needed to solve the problem **[1]**; example: representing a road network as a graph of nodes (junctions) and weighted edges (roads), ignoring scenery/buildings **[1]**. - Abstraction by generalisation/categorisation = grouping things by shared characteristics so they can be handled the same way **[1]**; example: treating every car/driver as a generic "Vehicle/Driver" object with the same attributes (location, available) regardless of make or model **[1]**.

(b) [4] — two valid sub-problems, each with a sensible input and output (2 marks each): - e.g. *Locate nearby drivers*: input = passenger pickup coordinates; output = list of drivers within range. **[2]** - e.g. *Calculate fare estimate*: input = distance/time of route; output = estimated price. **[2]** - (Other valid: rank drivers by ETA, check driver availability, track journey progress.)

(c) [2]: - Concurrency must be considered because many users (passengers/drivers) act on shared data simultaneously / thousands of requests at once **[1]**. - Problem: a **race condition** — both passengers could be matched to the same driver because each read "available" before the other updated it, leading to a double booking / inconsistent state **[1]**.

(d) [2]: - Benefit: faster response / less repeated computation as a stored result is returned immediately for a common route **[1]**. - Drawback: the cached estimate may be **out of date / inaccurate** when road/traffic conditions change, so fares are wrong until the cache is refreshed **[1]**.

Question 2 — Programming techniques [11]

(a) [1] — A function **returns a value** (to the calling expression); a procedure does not return a value / performs an action. **[1]**

(b) [2]: - By value: a **copy** of the data is passed; changes inside the subroutine do **not** affect the original variable. **[1]** - By reference: the subroutine receives a reference/pointer to the original; changes inside the subroutine **do** affect the original variable. **[1]**

(c) [3] — Output, in order:

```
5
11
```

- `x` is passed **by value**, so `tweak` works on a copy; `x` remains 5. **[1]** (prints `5`)
- `y` is passed **by reference**, so `b = b + 1` updates the original; `y` becomes 11. **[1]** (prints `11`)
- Correct ordering / both lines correct with reasoning. **[1]**

(d) [3]: - Scope = the region/part of the program in which a variable is **accessible / has meaning / exists**. **[1]** - (Allow: local scope = within the subroutine it is declared in; global = whole program.) **[1]** - Advantage of local variables: avoid unintended side-effects / name clashes / make subroutines self-contained and reusable / memory freed when subroutine ends. **[1]** (any one)

(e) [2]:

```
if age >= 18 AND hasLicence == true then
    print("May drive")
endif
```

- Correct combined condition with `AND`. **[1]**
- Same body / behaviour preserved (allow `if age >= 18 AND hasLicence`). **[1]**

Question 3 — Recursion and the call stack [10]

(a) [2]: - A **base case** (stopping condition) that returns without further recursion. **[1]** - A **general/recursive case** that moves the input **towards** the base case. **[1]**

(b) [5] — `f(n) = n * f(n-2)`, base case `n <= 1` returns 1. Calls pushed:

```
f(7) -> needs f(5)
f(5) -> needs f(3)
f(3) -> needs f(1)
f(1) -> base case, returns 1
```

Mark allocation: - Correct chain of calls 7 → 5 → 3 → 1 pushed onto stack. **[2]** - Base case `f(1) = 1` identified. **[1]** - Unwinding with correct returns: **[2]** - `f(1)` returns `1` - `f(3)` returns `3 * 1 = 3` - `f(5)` returns `5 * 3 = 15` - `f(7)` returns `7 * 15 = 105`

Stack table (for reference):

Frame pushed	Evaluates	Returns
f(7)	7 * f(5)	105
f(5)	5 * f(3)	15
f(3)	3 * f(1)	3
f(1)	base case	1

(c) [1] — `f(7) = 105`. **[1]**

(d) [2]: - With base case `n == 0`, the sequence 7 → 5 → 3 → 1 → −1 → −3 … **never equals 0** (it skips 0), so the recursion never reaches the base case / does not terminate. **[1]** - The call stack grows until memory is exhausted: a **stack overflow** error. **[1]**

Question 4 — Object-oriented programming [14]

(a) [5] — e.g.:

```
class Book
    private title
    private author
    private available

    public procedure new(t, a)
        title = t
        author = a
        available = true
    endprocedure
endclass
```

- Class declared with name `Book`. **[1]**
- Three attributes declared `private`. **[1]**
- Constructor takes `title` and `author` parameters. **[1]**
- Constructor assigns `title` and `author` from parameters. **[1]**
- Constructor sets `available = true`. **[1]**

(b) [4]:

```
public function borrowItem()
    if available == true then
        available = false
        return true
    else
        return false
    endif
endfunction

public procedure returnItem()
    available = true
endprocedure
```

- `borrowItem` checks `available` and, if true, sets it false and returns `true`. **[2]** (1 for condition/set, 1 for returns)
- `borrowItem` returns `false` when not available. **[1]**
- `returnItem` sets `available = true`. **[1]**

(c) [3]:

```
class ReferenceBook inherits Book
    public function borrowItem()
        return false
    endfunction
endclass
```

- Correct inheritance syntax (`inherits Book` / `extends`). **[1]**
- Overrides `borrowItem()` (same method name/signature). **[1]**
- Method always returns `false`. **[1]**

(d) [2]: - Concept = **encapsulation** (information hiding). **[1]** - Advantage: protects data integrity / state can only change via controlled methods / internal implementation can change without affecting other code. **[1]** (any one)

Question 5 — Computational methods [12]

(a) [2]: - Method = **backtracking**. **[1]** - Conceptual data structure = a **stack** (choices remembered/undone LIFO; recursion's call stack). **[1]**

(b) [3]: - Optimal algorithm guarantees the best possible solution but may be slow/exhaustive. **[1]** - Heuristic uses a "rule of thumb" to find a **good-enough** solution quickly, without guaranteeing optimality. **[1]** - Reason preferred for RidePool matching: must respond in **real time** to thousands of requests — a near-optimal match found instantly is more useful than the perfect match found too late. **[1]**

(c) [3]: - Divide and conquer = **break the problem into smaller sub-problems**, solve each (often recursively), then **combine** the results. **[1]** - In merge sort: repeatedly **split** the list in half until lists of length 1 (trivially sorted). **[1]** - Then **merge** adjacent sublists back together in order to build the sorted list. **[1]**

(d) [2]: - Method = **data mining** (finding patterns/associations in large data). **[1]** - Visualisation: e.g. a **graph/network diagram** of co-purchased products, a heat map, or a bar chart of association strength — used to decide product placement/promotions. **[1]**

(e) [2]: - A valid intractable/NP example, e.g. the **Travelling Salesman Problem** / optimal routing visiting all points. **[1]** - Justification: the number of possible solutions grows **factorially/exponentially**, so an exact solution is not computable in reasonable time for large n; a heuristic gives a good route quickly. **[1]**

Question 6 — Big O analysis [10]

(a) [2]: - Big O expresses how the **run-time (or space)** of an algorithm grows as a function of input size n / describes its worst-case order of growth. **[1]** - Lower-order terms and constants are discarded because, for **large n**, the highest-order term dominates the growth rate; we care about the rate of growth, not exact counts/hardware. **[1]**

(b) [4] — 1 mark each: - (i) Binary search: **O(log n)**. **[1]** - (ii) Bubble sort: **$O(n^2)$**. **[1]** - (iii) Stack push (array): **O(1)**. **[1]** - (iv) Doubling calls: **$O(2^n)$**. **[1]**

(c) [2]: - Complexity = **$O(n^2)$**. **[1]** - Justification: an inner loop of n iterations runs once for **each** of the n outer iterations → n × n = n^2 operations. **[1]**

(d) [2]: - For large n, n log n grows much more slowly than n^2, so the O(n log n) algorithm performs far fewer operations / scales better. **[1]** - Example O(n log n) average-case sort: **merge sort** (or quick sort). **[1]**

Question 7 — Searching algorithms [11]

(a) [1] — The list must be **sorted** (in order). **[1]**

(b) [5] — Searching for 44 in `[3,8,15,22,31,44,58,67,90]` (indices 0–8):

Step	low	high	mid	list[mid]	Action
1	0	8	4	31	44 > 31 → search right, low = 5
2	5	8	6	58	44 < 58 → search left, high = 5
3	5	5	5	44	**found**

Marks: - Step 1 correct (mid = 4, list[mid] = 31, go right). **[1]** - Step 2 correct (mid = 6, list[mid] = 58, go left). **[1]** - Step 3 correct (mid = 5, list[mid] = 44, found). **[1]** - Correct mid calculation each time (integer division of (low+high)/2). **[1]** - Number of comparisons of `list[mid]` = **3**. **[1]**

(Accept `mid = floor((low+high)/2)`. If a candidate rounds (5+8)/2 to mid=6 at step 2 that is correct; mid = floor(6.5)=6.)

(c) [3]:

```
function linearSearch(list, target)
    for i = 0 to list.length - 1
        if list[i] == target then
            return i
        endif
    next i
    return -1
endfunction
```

- Comparison `list[i] == target` inside loop. **[1]**
- `return i` when found. **[1]**
- `return -1` after loop (not found). **[1]**

(d) [2] — any one valid scenario (2 marks for a clearly justified one, or 1+1): - The data is **unsorted** and will only be searched once, so the cost of sorting it first isn't worth it. **[1]** - The list is **very small**, so the simplicity of linear search outweighs binary search's advantage. **[1]** - (Allow: data stored in a structure with no random access, e.g. a linked list, where binary search can't jump to the middle.)

Question 8 — Sorting algorithms [13]

(a) [4] — Insertion sort of `[5, 2, 8, 1, 4]`. The sorted portion grows from the left; show list after each insertion:

After inserting	List state
2 (into [5])	`[2, 5, 8, 1, 4]`
8 (already in place)	`[2, 5, 8, 1, 4]`
1 (to front)	`[1, 2, 5, 8, 4]`
4 (between 2 and 5)	`[1, 2, 4, 5, 8]`

- `[2,5,8,1,4]` after inserting 2. **[1]**
- `[2,5,8,1,4]` after 8 (unchanged, accept stated). **[1]**
- `[1,2,5,8,4]` after inserting 1. **[1]**
- `[1,2,4,5,8]` after inserting 4 (final sorted). **[1]**

(b) [3]: - Worst case: **O(n²)**. **[1]** - Best case: **O(n)**. **[1]** - Best case occurs when the input is **already sorted** (each element only compared once, no shifts). **[1]**

(c) [4] — Merge sort of `[38, 27, 43, 3, 9, 82, 10]`.

Divide:

```
[38, 27, 43, 3, 9, 82, 10]
[38, 27, 43, 3]            [9, 82, 10]
[38, 27]   [43, 3]         [9, 82]  [10]
[38][27]   [43][3]         [9][82]  [10]
```

Merge (conquer):

```
[27, 38]   [3, 43]         [9, 82]  [10]
[3, 27, 38, 43]            [9, 10, 82]
[3, 9, 10, 27, 38, 43, 82]
```

- Correct divide phase down to single elements. **[1]**
- Correct first level of merges (`[27,38]`, `[3,43]`, `[9,82]`, `[10]`). **[1]**
- Correct second level (`[3,27,38,43]` and `[9,10,82]`). **[1]**
- Correct final merge `[3,9,10,27,38,43,82]`. **[1]**

(d) [2]: - Merge sort is O(n log n) worst case vs bubble sort O(n²); it splits the work logarithmically (divide and conquer) so it scales far better on large lists. **[1]** - Disadvantage: merge sort needs **extra memory** (O(n)) for the temporary merged lists, whereas bubble/insertion sort are **in-place** (O(1) extra). **[1]**

Question 9 — Data structure algorithms [12]

(a) [5]:

```
procedure push(value)
    if top == MAX - 1 then
        print("Stack overflow")
    else
        top = top + 1
        stack[top] = value
    endif
endprocedure

function pop()
    if top == -1 then
        return "Stack underflow"
    else
        value = stack[top]
        top = top - 1
        return value
    endif
endfunction
```

- `push`: `top = top + 1`. **[1]**
- `push`: `stack[top] = value`. **[1]**
- `pop`: store `stack[top]` before decrementing. **[1]**
- `pop`: `top = top - 1`. **[1]**
- `pop`: returns the popped value. **[1]**

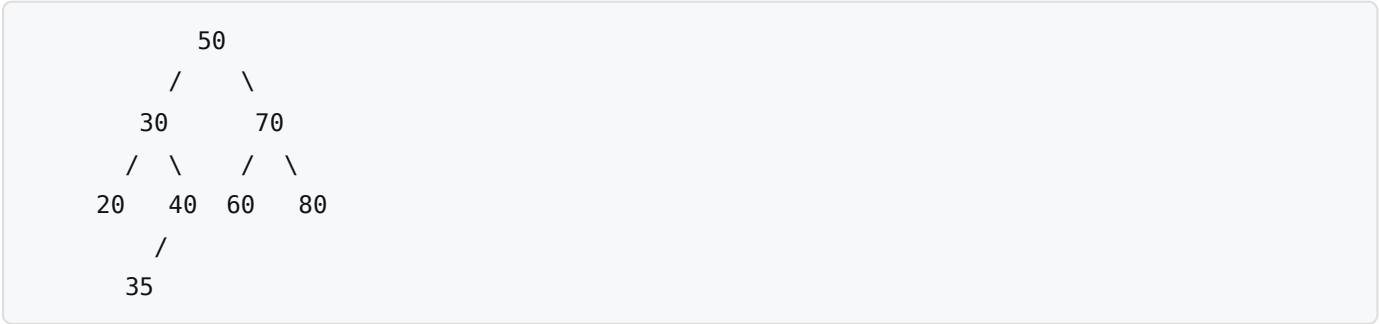
(b) [2]: - A circular queue **reuses** array slots vacated at the front (rear wraps around to index 0). **[1]** - A linear queue would leave the freed front slots unusable, wasting space / "drifting" and reporting full when slots are actually free; the circular queue uses the fixed buffer efficiently. **[1]**

(c)(i) [2] — Traverse from head (index 1): - 1 ("Beta") → 3 ("Alpha") → 4 ("Cati") → 2 ("Delta"). **[1]** - Output order: **Beta, Alpha, Cati, Delta**. **[1]**

(c)(ii) [3] — Insert "Echo" (index 5) between "Alpha" (index 3) and "Cati" (index 4): - New node index 5: data "Echo", pointer = **4** (points to "Cati"). **[1]** - Node 3 ("Alpha") pointer changes from 4 to **5**. **[1]** - All other pointers unchanged; new traversal: Beta, Alpha, Echo, Cati, Delta. **[1]**

Question 10 — Trees and traversals [11]

(a) [3] — Insert order `50, 30, 70, 20, 40, 60, 80, 35`:



- Root 50 with 30 (left) and 70 (right) correct. **[1]**
- 20 and 40 placed as children of 30; 60 and 80 as children of 70. **[1]**
- 35 placed as **left child of 40** (35 > 30 → right of 30 to 40; 35 < 40 → left of 40). **[1]**

(b) [4]: - **Pre-order** (root, left, right): `50, 30, 20, 40, 35, 70, 60, 80`. **[1]** - **In-order** (left, root, right): `20, 30, 35, 40, 50, 60, 70, 80`. **[1]** - **Post-order** (left, right, root): `20, 35, 40, 30, 60, 80, 70, 50`. **[1]** - All three fully correct (consistency/accuracy mark). **[1]**

(c) [2]: - In-order traversal of a BST outputs the values in **ascending sorted order**. **[1]** - Use: producing a sorted list / "tree sort" without a separate sort step. **[1]**

(d) [2]: - Worst-case search = **O(n)**. **[1]** - Caused when the tree is **unbalanced / degenerate**, effectively a linked list (e.g. inserting already-sorted data). **[1]**

Question 11 — Shortest path (Dijkstra / A*) [12]

Graph edges (undirected): A-B=2, A-C=5, B-C=1, B-D=7, C-D=3, C-E=6, D-E=1.

(a) [7] — Dijkstra from A. Tentative distances (∞ = unknown); finalise the unvisited node with the smallest tentative distance each round.

Visited (finalised)	A	B	C	D	E
start	0	∞	∞	∞	∞
A (0)	0	2 (via A)	5 (via A)	∞	∞
B (2)	0	2	3 (via B: 2+1)	9 (via B: 2+7)	∞
C (3)	0	2	3	6 (via C: 3+3)	9 (via C: 3+6)
D (6)	0	2	3	6	7 (via D: 6+1)
E (7)	0	2	3	6	7

Order finalised: **A, B, C, D, E.**

- Shortest distance A → E = **7**. **[1]**
- Shortest path = **A → B → C → D → E**. **[1]**

Marks: - Initialise A = 0, others ∞; relax A's neighbours (B=2, C=5). **[1]** - Finalise B next; update C to 3 (2+1) and D to 9 (2+7). **[1]** - Finalise C (=3); update D to 6 (3+3) and E to 9 (3+6). **[1]** - Finalise D (=6); update E to 7 (6+1). **[1]** - Correct order of visiting A, B, C, D, E shown. **[1]** - Correct final distance **7**. **[1]** - Correct path **A→B→C→D→E** (traced back via predecessors). **[1]**

(Total 7: visiting/working marks + distance + path.)

(b) [3]: - A* orders nodes by **f(n) = g(n) + h(n)**, where g(n) = actual cost from start to n and h(n) = heuristic estimate of remaining cost from n to goal. **[1]** (formula) **+[1]** (meaning of g and h) - Advantage: the heuristic **guides** the search towards the goal, so A* typically explores **fewer nodes** than Dijkstra (which expands outward in all directions), making it faster when a good heuristic exists. **[1]**

(c) [2]: - The heuristic must be **admissible** — it must **never overestimate** the true remaining cost to the goal. **[1]** - (Accept also: must be consistent/monotonic.) Explanation that an admissible heuristic guarantees the optimal path. **[1]**

Question 12 — Design and evaluation [12]

Level-of-response marking (0–12). Award the band that best fits, then a mark within it.

- **Level 3 (9–12):** A well-structured response that addresses **all four** required points. Data structure choice is justified, the searching approach is appropriate and explained with reference to relevant computational methods/techniques, time complexity is analysed correctly as drivers grow, and at least one trade-off is evaluated with a reasoned judgement. Technically accurate and uses correct terminology throughout. QWC strong.
- **Level 2 (5–8):** Addresses most points with some justification. Data structure and search approach identified and mostly appropriate; some complexity discussion; a trade-off mentioned but evaluation limited. Mostly accurate, reasonable structure.
- **Level 1 (1–4):** Fragmentary/descriptive. Names a structure and/or search method with little justification; little or no complexity analysis or evaluation. Some inaccuracies.
- **0 marks:** nothing creditworthy.

Indicative content (credit any valid, well-argued combination):

Data structures: - A **spatial structure** (e.g. a grid/buckets keyed by geographic cell, a quadtree, or a k-d tree) so that "nearest driver" queries only examine drivers in nearby cells rather than all drivers. - Within each cell, drivers held in a list/array; availability flag per driver. A **hash table** keyed by driver ID for O(1) status updates as drivers come on/off shift. - A **priority queue / min-heap** is appropriate if results are ranked by distance/ETA. - Alternatively model the road network as a **weighted graph** and use a shortest-path algorithm for true travel-time nearest driver.

Searching/algorithmic approach: - For straight-line nearest: query the spatial structure for candidate drivers in the pickup cell and adjacent cells, then compute distances and pick the minimum (a localised linear scan of few candidates). - For travel-time nearest on roads: **Dijkstra / A*** from the pickup point; A* with a straight-line distance heuristic (admissible) reduces nodes explored. - A **heuristic** (nearest in straight-line distance) may be accepted as "good enough" in real time rather than computing exact road distance to every driver.

Time complexity: - Naïve linear scan of all drivers = **O(n)** per request → with thousands of requests/sec and many drivers this is too slow. - A spatial index reduces the search to drivers in nearby cells, roughly **O(k)** where $k \ll n$ (drivers near the pickup), or **O(log n)** for tree-based structures — far more scalable as n grows. - Updates (driver on/off shift) should be O(1) or O(log n) so frequent changes don't bottleneck.

Trade-offs to evaluate: - **Speed vs accuracy:** a straight-line heuristic is fast but ignores one-way roads/traffic; exact routing is accurate but costly. - **Speed vs memory:** spatial indexes/caches use extra memory to gain query speed. - **Responsiveness vs optimality:** in real time, returning a near-optimal driver instantly beats the perfect driver found too late (links to heuristic justification). - **Concurrency:** updates and queries happen simultaneously — must avoid race conditions when marking a driver as taken (links to Q1c).

A response reaching Level 3 makes a clear recommendation (e.g. spatial grid + hash table for status + A* for travel time) and justifies it against these trade-offs.

Verified mark total

Question	Marks
Q1 Computational thinking	12
Q2 Programming techniques	11
Q3 Recursion & call stack	10
Q4 Object-oriented programming	14
Q5 Computational methods	12
Q6 Big O analysis	10
Q7 Searching algorithms	11
Q8 Sorting algorithms	13
Q9 Data structure algorithms	12
Q10 Trees and traversals	11
Q11 Shortest path (Dijkstra / A*)	12
Q12 Design and evaluation	12
TOTAL	**140**

Verified total = 140 marks.